The login system described in this chapter is pretty basic, but sufficient for many applications. Applications with more complex requirements generally use a gem to do this.

A number of plugins are available that provide ready-made solutions that are more comprehensive than the authentication logic shown here and often don't require significantly more code and effort on your part to use. Devise[81] is a common and popular gem that does this.

What We Just Did

By the end of this iteration, we've done the following:

- We used `has_secure_password` to store an encrypted version of the password into the database.
- We controlled access to the administration functions using before action callbacks to invoke an `authorize` method.
- We used `rails console` to interact directly with a model (and dig us out of a hole after we deleted the last user).
- We used a transaction to help prevent deletion of the last user.

Playtime

Here's some stuff to try on your own:

- Modify the user update function to require and validate the current password before allowing a user's password to be changed.
- The system test in `test/system/users_test.rb` was generated by the scaffolding generator we used at the start of the chapter. Those tests don't pass. See if you can get them to pass without breaking the other system tests. You'll recall we created the module `AuthenticationHelpers` and included it in all of the system tests by default, so you might need